

VeriGen: A Large Language Model for Verilog Code Generation

SHAILJA THAKUR, New York University, New York, USA

BALEEKGH AHMAD, New York University, New York, USA

HAMMOND PEARCE, University of New South Wales, Sydney, Australia

BENJAMIN TAN, University of Calgary, Calgary, Canada

BRENDAN DOLAN-GAVITT, New York University, New York, USA

RAMESH KARRI, New York University, New York, USA

SIDDHARTH GARG, New York University, New York, USA

In this study, we explore the capability of Large Language Models (LLMs) to automate hardware design by automatically completing partial Verilog code, a common language for designing and modeling digital systems. We fine-tune pre-existing LLMs on Verilog datasets compiled from GitHub and Verilog textbooks. We evaluate the functional correctness of the generated Verilog code using a specially designed test suite, featuring a custom problem set and testing benches. Here, our fine-tuned open-source CodeGen-16B model outperforms the commercial state-of-the-art GPT-3.5-turbo model with a 1.1% overall increase. Upon testing with a more diverse and complex problem set, we find that the fine-tuned model shows competitive performance against state-of-the-art gpt-3.5-turbo, excelling in certain scenarios. Notably, it demonstrates a 41% improvement in generating syntactically correct Verilog code across various problem categories compared to its pre-trained counterpart, highlighting the potential of smaller, in-house LLMs in hardware design automation. We release our training/evaluation scripts and LLM checkpoints as open-source contributions.

CCS Concepts: • **Hardware** → **Hardware description languages and compilation**; • **Computing methodologies** → **Natural language processing**; **Machine translation**;

Additional Key Words and Phrases: Transformers, verilog, GPT, large language models, EDA

ACM Reference Format:

Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2024. VeriGen: A Large Language Model for Verilog Code Generation. *ACM Trans. Des. Autom. Electron. Syst.* 29, 3, Article 46 (April 2024), 31 pages. <https://doi.org/10.1145/3643681>

This manuscript extends work presented at the Design, Automation & Test in Europe Conference (DATE 2023) [31].

This research work was supported in part by NSF Award 1553419, NSF Award 1646671, NSF Award 2039607, and ARO Award 77191NC. The opinions, findings, and conclusions, or recommendations expressed are those of the author(s) and do not necessarily reflect the views of any sponsors.

Authors' addresses: S. Thakur (Corresponding author), B. Ahmad, B. Dolan-Gavitt, R. Karri, and S. Garg, New York University, 6 MetroTech Center, New York, NY 11201, USA; e-mails: st4920@nyu.edu, ba1283@nyu.edu, brendandg@nyu.edu, rkarr@nyu.edu, siddharth.garg@nyu.edu; H. Pearce, University of New South Wales, Sydney, NSW 2052, Australia; e-mail: hp2265@nyu.edu; B. Tan, University of Calgary, 2500 University Drive NW, Calgary Alberta T2N 1N4, CANADA; e-mail: benjamin.tan1@ucalgary.ca.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1084-4309/2024/04-ART46

<https://doi.org/10.1145/3643681>

1 INTRODUCTION

Digital hardware design flows involve designers writing code in **hardware description languages (HDLs)** such as Verilog and VHDL to specify hardware architectures and behaviors, a process that is both time-consuming and bug-prone [10]. As design complexity grows, there is a need to reduce design costs and developer effort during hardware specification. Several attempts have thus sought to improve HDL design time and quality, for instance by using high-level synthesis—this allows developers to specify functionality in languages like C but comes at the expense of hardware efficiency. Related works also consider modernizing HDLs by adopting features and languages typically used in software development, for instance Chisel [4] based on Scala.

A promising new approach comes via the proliferation of technically capable code-writing **large language models (LLMs)** [8]. LLMs are deep neural networks, typically based on transformer [32] architectures, that aim to model the underlying distribution of a natural or structured language corpus. Given a sequence of words (or “tokens”) LLMs predict a distribution over the next word/token. When used in a loop (“autoregressively”) with some “input prompt” and strategies for choosing the best tokens from the distribution, LLMs can thus complete text: If prompted with the first sentence of a paragraph of English prose, the model will suggest more text; or, if provided technical specifications or coding comments, LLMs can then implement the matching code. The proficiency of these pre-trained models are a result of recent advance in scaling transformer architectures as well as the availability of massive text and code corpus from open-source repositories such as GitHub and StackOverflow. As such, we wish to investigate the potential for developers to use LLMs to automatically generate HDL code directly from a natural language like English.

While considerable effort has been undertaken which explores the training and utilization of language models for writing software [1, 8, 18] as well as in their down-stream effects (e.g., security [23, 29]), there is scant examination of LLMs for hardware applications. As such, this was the focus of the original work [31] as well as in this extension, where we perform the first comprehensive evaluation of the syntactic and functional correctness of synthesizable Verilog code generated by both open-source and commercial LLMs.

There are several key observations when considering the state-of-the-art in code-writing LLMs for hardware. First, it has been observed that existing commercial LLMs, including GitHub Copilot, may generate Verilog code which fails syntax, synthesis, and functional checks [23]. Second, while fine-tuning a given LLM over a Verilog corpus can aid in its authorship of HDL [24], this requires a large dataset of Verilog. Unfortunately, such datasets for Verilog are lacking, and so the prior work in this area used a small, synthetic template-generated HDL corpus meaning that the resultant model did not generalize to unseen problems. Third, quantifying model performance in this area is difficult: while datasets like HumanEval exist for software [8], large-scale test problems and methods to evaluate the syntactical and functional correctness of LLM-generated HDL are lacking.

Our article thus makes the following contributions. (1) By consolidating available open-source Verilog code we create (to the best of our knowledge) the largest training corpus of Verilog code yet used for training LLMs. (2) Using this corpus, we examine fine-tuning five different pre-trained LLMs models with parameter counts ranging from 345M to 16B, producing five new fine-tuned open-source models specialized for Verilog code completion. (3) To evaluate the efficacy of the models and determine the effect of the parameter sizes, we design a set of Verilog coding problems with varying difficulty levels and corresponding test benches to test the functional correctness of completed code. (4) We comprehensively evaluate our fine-tuned models against state-of-the-art general-purpose LLMs including ChatGPT (GPT-3.5-turbo and GPT4) and PALM2 an expanded evaluation with a new set of Verilog design challenges. These LLMs are several times larger than our models, black-boxed and incur per-token costs; in contrast, our models are smaller,

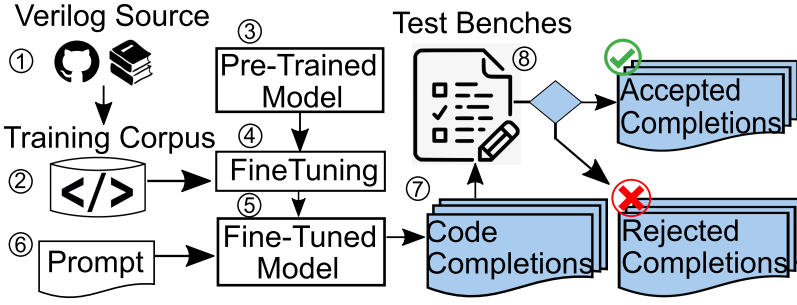


Fig. 1. Experimental evaluation of LLM Verilog completions.

open-sourced, free, and provide comparable or improved performance. (5) We augment our training set with a selection of Verilog textbooks and provide detailed results on the performance of models tuned on this augmented dataset. To aid the community in performing further research in this area, we provide the training/evaluation code¹ and LLM checkpoints.²

Figure 1 illustrates our experimental platform for studying the impact of parameters such as temperature, number of sequences generated per problem, and number of LLM parameters. Section 3 discusses creating the training data from GitHub and PDFs of Verilog textbooks (1) with pre-processing (2) and the five pre-trained LLMs (3) that we fine-tune (4) for completing Verilog code (5). Section 4 explains the evaluation setup, including our hand-designed prompts (6). Section 5 presents our results from generating code suggestions (7) and evaluating with an analysis pipeline that compiles the Verilog and checks it against unit tests (8). Section 7 discusses how our evaluation shows that the largest code-based LLMs (i.e., CodeGen-16B) fine-tuned on our Verilog corpus demonstrate competitive performance over all other evaluated LLMs. Qualitatively, our best-performing fine-tuned LLMs can generate functioning code for challenging problems.

2 BACKGROUND AND RELATED WORK

2.1 Background

Transformer-based deep neural networks have been the cornerstone of advances across domains excelling in language-related tasks [8, 32]. Development and use of Large Language Models (LLMs) is a substantial evolution in this landscape. LLMs process inputs formatted as tokens.³ When an input sequence of tokens known as a *prompt* is provided, the LLM processes it to generate a probability distribution for the next token across the entire vocabulary. The token with the highest probability is chosen, appended to the input sequence, and re-introduced into the LLM. This yields a new token, and the process iterates until a complete sequence of tokens, known as a *completion*, is generated.

In the domain of coding, LLMs exhibit a unique and innovative approach. In addition to natural-language, LLMs are trained on extensive code bases, either in a specific programming language or a mix of languages. The datasets used for these training tasks can often reach into hundreds of gigabytes. The prompts used for these code-trained LLMs can take various forms, including instructions, comments, code snippets, or a combination of these. Interestingly an LLM trained on a corpus of mixed languages often manages to infer the target language from the prompt.

¹<https://github.com/shailja-thakur/VGen>

²<https://huggingface.co/shailja>

³Tokens are common character sequences that have unique identification through byte pair encoding [14].

Despite the remarkable abilities of LLMs, training them from scratch is resource-intensive, requiring massive datasets and numerous parameters. However, there is a practical alternative of fine-tuning pre-trained LLMs on specialized datasets to cater to specific tasks. Fine-tuning is significantly efficient as it requires a limited number of training epochs. Several LLMs pre-trained for natural language and code either make the weights available, like NVIDIA’s MegatronLM [30] or Salesforce’s CodeGen models [21], or provide fine-tuning through an API, like AI21studio’s Jurassic-1 (J1) models⁴ [3]. However, fine-tuning through APIs may come with costs and lacks transparency as the models’ weights and parameters are not disclosed.

Recently, the LLM landscape has seen remarkable advancements in terms of larger and more complex models such as GPT-3.5-turbo [36], GPT-4 [22], PALM2 [9]. These LLMs have not only surpassed their predecessors in their capabilities but have achieved remarkable human task alignment. They have been trained on vast datasets, often comprising a large number of tokens, pushing the frontier of language-related tasks like code generation, debugging, and interpretation. These LLMs have achieved state-of-the-art results, showcasing the potential of machine learning in understanding and emulating human-like text generation.

However, these advances have brought new challenges. Larger models, unlike their open-source counterparts, come with usage fees. Furthermore, these commercial off-the-shelf products LLMs do not guarantee reliability or continuous presence in the public domain. For instance, Code-Davinci-002, a commercial LLM, is no longer accessible, highlighting risks of dependency on such models. Moreover, interfacing with these LLMs via APIs presents security vulnerabilities, such as the risk of injection attacks, where malicious commands could be inserted and executed. Furthermore, API usage may introduce additional latency, impacting the performance of applications that rely on these LLMs for real-time processing. We refer to these models that are supposedly hundreds of billions of parameters as *large* LLMs.

2.2 Prior Work

Programming is challenging, given the need for human designers to interpret and transform natural language specifications into programming structures. This motivates the use of **natural language processing (NLP)** to transform language to code [20]. Hardware design using Verilog HDL is similar to programming. Prior work explored NLP techniques for formal system modeling [12] and generating assertions [16], albeit on a small scale. Pearce et al. trained DAVE, a small LLM to produce Verilog snippets from template-based natural language descriptions for a limited set of functions [24]. GitHub’s Copilot was evaluated for security bugs produced during out-of-the-box Verilog completions [2, 23] and was found to be lacking.

Our study scales this exploration by assessing LLMs across various design tasks using an automated evaluation framework. Despite the lack of open datasets for Verilog-focused LLM training, the promising outcomes from applying LLMs in programming, and their further potential shown in multi-language code repositories like CodeSearchNet [17], and large-scale LLMs such as GPT-3.5 and GPT-4, emphasize the possibilities for hardware design languages. In addition, a study conducted by Blocklove et al. [5] showcases the possibility of applying ChatGPT for end-to-end synthesizable hardware design. As well, a series of studies [7, 13, 19] evaluating the potential of pre-trained LLMs further demonstrate their capabilities for hardware design languages.

However, there is no open dataset available to train and evaluate LLMs specialized on writing Verilog code. Our work presents a comprehensive evaluation by (1) fine-tuning a range of code-based LLMs on a Verilog training corpus, and (2) evaluating these models on a range of about

⁴<https://studio.ai21.com/docs/jurassic1-language-models/#general-purpose-models>

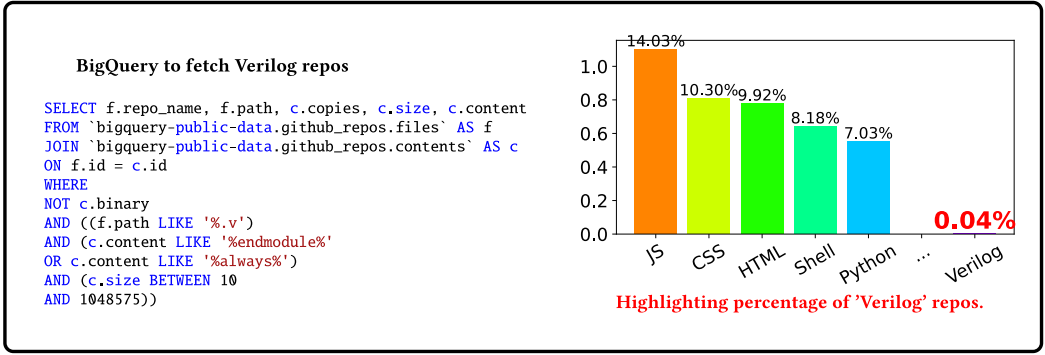


Fig. 2. BigQuery SQL query for fetching Verilog repositories with .v files of size less than 1,024 kB, accompanied by a comparative visualization of data volume across top programming languages on GitHub.

130 different Verilog problems. The evaluation utilizes both a handcrafted test set, refer Table 3 and a publicly available Verilog benchmark set [19].

3 LLM TRAINING

We describe our method for training (or fine-tuning) LLM models. We begin by describing our curated Verilog datasets, followed by the LLM architectures and fine-tuning method.

3.1 Verilog Training Corpus

Our primary Verilog training corpus comes from open-source Verilog code in public GitHub repositories. Additionally, we also created a dataset of text from Verilog textbooks to understand whether that further improved LLM performance.

GitHub Corpus. We use Google BigQuery to gather Verilog repositories from GitHub, where it has a snapshot of over 2.8 million repositories. We use a query that looks for keywords such as “Verilog” and files with “.v” extension as shown in Figure 2. Specifically, we query for non-binary files with “.v” extensions that contain Verilog module definitions (the `endmodule` keyword) or always blocks (the `always` keyword). The file size is restricted to between 10 and 1,048,575 characters. This query allows extracting meaningful Verilog code, while filtering out very small or unusually large files.

We de-duplicated files using MinHash and Jaccard similarity metrics [35] to remove duplicate Verilog code. Files were further filtered to only retain .v files containing at least one pair of `module` and `endmodule` statements. This is done to avoid retaining any .v file without any Verilog. The resulting corpus contains approximately 50,000 Verilog files with a size of 1 GB.

Additionally, we filtered out large files with over 20,000 characters. Restricting file lengths helps narrow the context window for LLMs during training. By keeping file sizes smaller than LLMs’ maximum context window, the models can effectively encode entire files allowing better generalization. The final training corpus after the filter is 400 MB, retaining smaller but high quality programs, representative examples ideal for fine-training tasks. In addition, this filter also helps eliminate very short and long examples which could have added to the skeweness of the dataset avoiding generalization issues. The efficiency of LLMs allows effective learning even from small datasets, facilitating better generalization capabilities.

Verilog Books Corpus. We downloaded 70 Verilog-based textbooks from an online e-library in PDF format, then extracted text using the Python-based tool `pymuPDF` which uses optical

character recognition to extract text. Depending on the quality of the PDF, the text quality varies. We encountered challenges including improper formatting of lines, unrecognized logical operators, and header/footer floating in-between text. We cleaned the text by filtering irrelevant passages (e.g., index, preface, and acknowledgments) and used regular expressions such as: `module(. *\\n*s*t*)\\((?!module)(?!endmodule). *\\W*)*endmodule` to identify blocks of prose with embedded Verilog snippets. From these blocks, overlapping sliding windows were used to produce training examples.

The PDFs were segmented into fixed length chunks (around 20,000 characters) to facilitate this extraction process. Importantly, we analyzed each chunk to ensure complete `module-endmodule` blocks were retained. Any incomplete blocks at chunk boundaries were reconstructed by parsing additional context from adjacent chunks. Avoiding truncated code blocks reduces ambiguity during pre-training given the corpus size.

The final combined corpus of textbook-extracted and GitHub code comprises 400 MB of text. The care taken in chunking and reconstructing intact blocks improves integrity of examples for the LLMs to learn prototypical Verilog structures effectively.

3.2 Baseline LLM Architectures

Table 1 shows the LLMs we used and summarizes design parameters, including the number of layers, heads, embedding size (head dimension), context length, and the data source (natural language (NL) and/or code). As code-davinci-002 is derived from GPT-3 [8], it has the same architecture. Since its parameters are not known, so we leave these as NA.

Additionally, Table 2 summarizes other practical characteristics of the models including open-source availability, cost per 1,000 tokens, presence of rate limits, and Internet connectivity requirements during inference. For instance, OpenAI's GPT-3.5 and GPT4 models have usage-based pricing models costing 0.002 and 0.06 per 1,000 tokens respectively. They also rate limit requests. In contrast, PALM from Google is currently free but also rate limited as well strict filters at the output limits the models. The open-source Megatron, CodeGen, and GPT-3.5-turbo models can be freely deployed on local infrastructure without rate limits or Internet connection requirements.

Understanding these tradeoffs aids selection for real-world applications. Overall, our evaluation aims to benchmark performance across diverse LLMs spanning proprietary and open-source domains and ranging from small to massive scale pre-training. The architectures and accessibility details provide useful perspective when interpreting downstream task efficacy.

3.3 LLM Fine-Tuning

We fine-tune five LLMs from Table 1 on our Verilog training datasets. Training the CodeGen LLMs was challenging due to the large number of parameters. At 16-bit precision, the parameters of CodeGen-16B alone require around 30 GB of GPU memory. Furthermore, the fine-tuning process requires even more GPU memory to store the intermediate computations and optimizer states, totaling around 250 GB. This necessitates the use of multiple GPUs for successful operation. For instance, in our case, we relied on 3 A100 GPUs to train this model for just one epoch. We use model and data parallelism and strategies for sharding the optimizer states across GPUs similar to DeepSpeed⁵ [27] and ZeRO [28].

We set the training hyperparameters to their defaults as recommended for the ZeRO-3 [26] optimizer state implementation. The deepspeed configuration details used in fine-tuning the selected LLMs can be accessed from our open-source GitHub repository.⁶ The CodeGen LLMs (2B, 6B, 16B)

⁵https://huggingface.co/docs/transformers/main_classes/deepspeed

⁶<https://github.com/shailja-thakur/CodeGen-Fine-Tuning>

Table 1. Baseline LLM Architectures Used in Our Study

Model-Parameters/Pre-Training Data	Layers	Heads	Embed.	Context Length
MegatronLM-355M [30]/NL [11, 25]	24	16	64	1024
J1-Large-7B ¹ /NL [6]	32	32	128	4096
CodeGen-2B [21]/NL [15], Code	32	32	80	2048
CodeGen-6B/NL [15], Code	33	16	256	2048
CodeGen-16B/NL [15], Code	34	24	256	2048
code-davinci-002 [8]/NL [6], Code	NA	NA	NA	8001
GPT-3.5-turbo [36]/NL [6], Code	NA	NA	NA	4096
GPT4 [22]/NL, Code	NA	NA	NA	8000
PALM2 [9]/NL, Code	NA	NA	NA	8000

Table 2. Baseline LLM Architectures Used in Our Study with Details on the Ease of Accessibility and Resource Intensive

Model-Parameters/Pre-Training Data	Open-Source	Cost/1k Token	Rate Limited?	Internet Required?
MegatronLM-355M [30]/NL [11, 25]	Yes	Free	No	No
J1-Large-7B ¹ /NL [6]	No	0.0001	Yes	Yes
CodeGen-2B [21]/NL [15], Code	Yes	Free	No	No
CodeGen-6B/NL [15], Code	Yes	Free	No	No
CodeGen-16B/NL [15], Code	Yes	Free	No	No
code-davinci-002 [8]/NL [6], Code	No	0.0020	Yes	Yes
GPT-3.5-turbo [36]/NL [6], Code	No	0.0020	Yes	Yes
GPT4 [22]/NL, Code	No	0.06	Yes	Yes
PALM2 [9]/NL, Code	No	Free	Yes	Yes

are fine-tuned for 1 epoch on an HPC cluster with two RTX8000s, four RTX8000s, and three A100s, and training completes in two, four, and six days, respectively. Megatron-LM is fine-tuned for 9 epochs using one RTX8000 for 15 hours using the default configuration [30]. We use the off-the-shelf AI21 studio for fine-tuning J1-Large.

4 LLM EVALUATION SETUP

To gauge the proficiency of Large Language Models (LLMs) in generating high quality Verilog code, we employ two distinct evaluation harnesses. The first harness, fully transparent in its operation, includes a hand-designed problem set (Set I, in Table 3), a collection of hand-designed test benches, and a comprehensive end-to-end pipeline to determine if the Verilog code, corresponding to a given prompt, adheres to the criteria of functional correctness.

Our second evaluation harness relies on a broader and more diversified set of problems (Set II, in Table 4) extracted from HDLBits, a popular digital logic learning platform [34] that allows users to test their solutions against a built-in test bench suite.

4.1 Problem Sets

Problem Set I is composed of 17 unique Verilog challenges, which have their roots in classroom exercises and examples from the HDLBits platform. Each problem has an assigned difficulty level,

Table 3. Problem Set I

Prob. #	Difficulty	Description
1	Basic	A simple wire
2	Basic	A 2-input and gate
3	Basic	A 3-bit priority encoder
4	Basic	A 2-input multiplexer
5	Intermediate	A half adder
6	Intermediate	A 1-to-12 counter
7	Intermediate	LFSR with taps at 3 and 5
8	Intermediate	FSM with two states
9	Intermediate	Shift left and rotate
10	Intermediate	Random Access Memory
11	Intermediate	Permutation
12	Intermediate	Truth table
13	Advanced	Signed 8-bit adder with overflow
14	Advanced	Counter with enable signal
15	Advanced	FSM to recognize “101”
16	Advanced	64-bit arithmetic shift register
17	Advanced	ABRO FSM*

*from Potop-Butucaru, Edwards, and Berry’s “Compiling Esterel”.

which is depicted in Table 3. These challenges cover a wide range of design concepts, including combinational and sequential logic designs, finite state machines with varying requirements, operations such as permutation, shift left, and rotate, and basic components like a multiplexer (MUX), Random Access Memory (RAM), Linear Feedback Shift Register (LFSR), adders, and counters. Examples of basic, intermediate, and advanced problems are presented in Figures 5–7, respectively. These have been obtained using CodeGen-16B-FT and have been edited for clarity and brevity.

Problem Set II significantly expands on this initial set by integrating an extensive range of problems from HDLBits [19, 34]. This expansion enriches our dataset to encompass a total of 181 problems. The enlarged dataset introduces new challenges distributed across four difficulty levels and multiple unique categories, each dedicated to a specific facet of hardware design and Verilog syntax. The dataset spans difficulty levels ranging from *Getting Started* to *Verilog Language* to *Circuits* and *Verify Bugs*, shown in Table 4. Within these levels, categories extend from basic principles such as gates to more advanced concepts like finite state machines and cellular automata, thus presenting a comprehensive view of hardware design and Verilog syntax.

4.2 LLM Inference

The LLM input is a prompt from the problem set in Section 4.1. We truncated completed code at keywords `end` and `endmodule` and pass the solution to the appropriate evaluation harness for checking its compilation and functional correctness.

4.2.1 Input Parameters. Each query to the LLM includes a prompt, a sampling temperature (t), and a number of completions per prompt (n).

Prompts: For the hand-designed Problem Set I, each problem is accompanied by three prompts of increasing detail: low (L), medium (M), and high (H). Prompt L has an initial comment describing the function of the module and the module header with name and inputs/outputs with types. We declare internal signals. M includes L plus comments that describe the function using signal names.

Table 4. Problem Set II - Extended Set of 164 Problems Derived from HDLBits [34]

Difficulty	Category	Count	Problem Description
Getting Started	Getting Started	2	Getting Started, Output Zero
Verilog	Basics	8	Simple/Four wires, Inverter, AND, NOR, XNOR, Declare wires, 7458 chip
	Vectors	9	Vectors, Vectors (detail), Vector part select, Bitwise operators, Four-input gates, Vector concatenate, Vector reversal 1, Replicate, More replication
	Module Hierarchy	9	Modules, Connect ports by position, Connect ports by name, Three modules, Modules and vectors, Adder 1, Adder 2, Carry-select, Adder-subtractor
	Procedures	8	Always blocks (combinational), Always blocks (clocked), If statement, If statement latches, Case statement, Priority encoder, Priority encoder with casez, Avoiding latches
	More Features	7	Conditional ternary, Reduction operators, Reduction: Wider gates, Combinational for-loop: Vector reversal 2, Combination for-loop: 255-bit count, Generate for-loop: 100-bit adder 2, Generate for-loop: 100-digit BCD adder
Circuits	Basic	17	Wire, GND, NOR, Another, Two gates, More gates, 7420 chip, Truth tables, Two-bit equality, Simple circuits A, B, Combine circuits A, B, Ring or vibrate?, Thermostat, 3-bit count, Gates and vectors, longer vectors
(Combination)	Multiplexers	5	2-to-1 mux, 2-to-1 bus mux, 9-to-1 mux, 256-to-1 mux, 256-to-1 4-bit mux
	Arithmetic Circuits	7	Half adder, Full adder, 3-bit adder, Signed addition overflow, 100-bit binary adder, 4-digit BCD adder
	K-Map to Circuit	8	3/4-variable, Minimum SOP and POS, K-map, K-map with a mux
Circuits (Sequential)	Latches and Flip-Flops	18	DFFs, DFF (reset), DFF (reset value), DFF (asynchronous), DFF (byte enable), D Latch, DFF, DFF+gate, Mux and DFF, DFFs and gates, Circuit from truth table, Detect edge/both edges, Edge capture register, Dual-edge triggered FF
	Counters	8	Four-bit binary counter, Decade counter, Decade counter again, Slow decade counter, Counter 1-12, Counter 1000, 4-digit decimal counter, 12-hour clock
	Shift Registers	9	4-bit shift register, Left/right rotator, Left/right arithmetic shift by 1 or 8, 5-bit LFSR, 3-bit LFSR, 32-bit LFSR, Shift register, Shift register, 3-input LUT
	Cellular Automata	3	Rule 90, Rule 110, Conways Game of Life 16x16
	FSM	33	FSM 1 (asynchronous), FSM 1 (synchronous), FSM 2 (asynchronous), FSM 2 (synchronous), Simple state transitions 3, Simple one-hot state transition 3, FSM 3 (asynchronous), FSM 3 (synchronous), Moore FSM, Lemmings 1, Lemmings 2, Lemmings 3, Lemmings 4, One-hot FSM, PS/2 packet parser, PS/2 packet parser and datapath, Serial receiver, Serial receiver and datapath, Serial receiver with parity check, Sequence recognition, Q8: Design Mealy FSM, Q5a: Serial twos complementer (Moore FSM), Q5b: Serial twos complementer (Mealy FSM), Q2a, Q2b, Q3a, Q3b: FSM, Q3c: FSM logic, Q6b: FSM next-state logic, Q6c: FSM one-hot next-state logic, Q6: FSM, Q2a: FSM, Q2b: One-hot FSM equations
	Larger Circuits	7	Counter with period 1000, 4-bit shift register and down counter, FSM: Sequence 1101 recognizer, FSM: Enable shift register, FSM: The complete FSM, The complete timer, FSM: One-hot logic equations
Verify Bugs	Read Simulations & Find bugs	5	Mux2, NAND, Mux4, Add/subtract, Case statement

Prompt:

```

1 // This is a finite state machine that recognizes the sequence 101 on the input signal x.
2 module adv_fsm(input clk, input reset, input x, output z);
3 reg [1:0] present_state, next_state;
4 parameter IDLE=0, S1=1, S10=2, S101=3;
5 // output signal z is asserted to 1 when present_state is S101
6 // present_state is reset to IDLE when reset is high,
7 // otherwise it is assigned next state
8 // if present_state is IDLE, next_state is assigned S1 if
9 // x is 1, otherwise next_state stays at IDLE
10 // if present_state is S1, next_state is assigned S10 if
11 // x is 0, otherwise next_state stays at IDLE
12 // if present_state is S10, next_state is assigned S101 if
13 // x is 1, otherwise next_state stays at IDLE
14 // if present_state is S101, next_state is assigned IDLE

```

Fig. 3. Varying the prompt details: Low (L): 1-4, Medium (M): 1-8, and High (H): 1-14. Set I, Problem 15.

H replaces and/or appends comments in M with more detail and resembles pseudo-code instead of a predominantly natural language specification. Figure 3 is an example for Problem 15. L has no lines highlighted (the prompt is lines 1–4). M includes L and lines highlighted yellow (the prompt is lines 1–8). H includes M and lines in gray (the prompt is lines 1–15).

For Problem Set II, we formulate the prompts in alignment with the problem descriptions provided on the platform. A typical prompt for each problem starts with a set of comments, abstracted from the high-level problem description available on the platform. Following this is the module header, tagged as `top_module`, which includes inputs and outputs, each with its defined type. Finally, a comment line prompts the insertion of the problem-solving code.

An illustrative example of this format is depicted in Figure 4, applied to the *vibrate&ring* problem. This prompt features a detailed level of description. The lines 1-13 contain the initial comments, which describe the problem function. This section uses signal names and high-level specifications. The following segment (lines 15-20) includes the module header skeleton, which comprises the module’s name and input/output data. And line 22 holds a single line comment instructing the user to insert their code.

Sampling temperature (t): A higher value means that the LLM takes more risks and yields more creative completions. We use $t \in \{0.1, 0.3, 0.5, 0.7, 1\}$.

Completions per prompt (n): For each prompt, LLM generates n completions where $n \in \{1, 10, 25\}$. For J1-Large and PALM2, we skip $n = 25$ because they do not support this value. For GPT4, we only use $n = 1$ due to the high cost of usage.

max_tokens: The maximum number of tokens generated for each completion was set to 300 for all autocomplete language models (Megatron-LM, CodeGen, Code-davinci-002), except for J1-Large. J1-Large had a token limit of 256. We used the default nucleus sampling probability mass (`top_p`) setting. For larger models like GPT3.5, PALM2, and Claude, which generate responses in a conversational style, we increased the maximum tokens to 900 per completion, as these models typically consume more tokens compared to autocomplete ones. However, for GPT4, the web interface doesn’t provide an option to adjust the token limit.

4.3 Test Benches

For Problem Set I, we developed a test bench to validate the functional correctness of each solution. The test benches exercise the designs for corner cases and are exhaustive for basic and some intermediate cases. For the remaining cases, the test benches are analogous to unit tests. This keeps the evaluation time reasonable, e.g., for the RAM module, the data width is 8 and the address width

Prompt:

```

1 // Suppose you are designing a circuit to control a cellphone's ringer and vibration motor.
2 // Whenever the phone needs to ring from an incoming call (input ring), your circuit must either
3 // turn on the ringer (output ringer = 1) or the motor (output motor = 1), but not both.
4 // If the phone is in vibrate mode (input vibrate_mode = 1), turn on the motor. Otherwise, turn on the
  ringer.
5
6 // Try to use only assign statements, to see whether you can translate a problem description into a
  collection of logic gates.
7
8 // Design hint: When designing circuits, one often has to think of the problem "backwards",
9 // starting from the outputs then working backwards towards the inputs. This is often the opposite
10 // of how one would think about a (sequential, imperative) programming problem, where one would look
11 // at the inputs first then decide on an action (or output). For sequential programs, one would
12 // often think "If (inputs are ___ ) then (output should be ___)". On the other hand, hardware
13 // designers often think "The (output should be ___ ) when (inputs are ___)".
14
15 module top_module (
16     input ring,
17     input vibrate_mode,
18     output ringer, // Make sound
19     output motor   // Vibrate
20 );
21
22 // Insert your code here

```

Correct Completion:

```

1 assign ringer = (vibrate_mode == 0) ? ring : 0;
2 assign motor = (vibrate_mode == 1) ? ring : 0;
3
4 endmodule

```

Incorrect Completion:

```

1 // If the phone is in vibrate mode, turn on the
2 // motor. Otherwise, turn on the ringer.
3 assign motor = vibrate_mode;
4 assign ringer = (vibrate_mode) ? 0 : ring;
5
6 endmodule

```

Fig. 4. Set II, Vibrate & ring problem. Difficulty: Circuits (Combinational), Basic category. We highlight the mistake . motor turns on even with ringer set to 0 and vibrate_mode set to 1

Prompt:

```

1 // This is a 3-bit priority encoder. It outputs the position of the first high bit.
2 module priority_encoder(input[2:0] in,output reg[1:0] pos);
3 // If none of the input bits are high (i.e., input is zero), output zero.
4 // assign the position of the highest bit of in to pos.

```

Correct Completion:

```

1 always @(in)
2 if (in == 0) pos <= 2'h0;
3 else if (in[0]) pos <= 2'h0;
4 else if (in[1]) pos <= 2'h1;
5 else pos <= 2'h2;
6 endmodule

```

Incorrect Completion:

```

1 always @(in) begin
2 case(in)
3 3'b000: pos=2'b00;
4 3'b001: pos=2'b01;
5 3'b010: pos=2'b10;
6 3'b011: pos=2'b11;
7 default: pos=2'b00;
8 . . .

```

Fig. 5. Set I, Basic example - Problem 3: A 3-bit priority encoder. We highlight the mistake . Positions are offset by 1.

is 6 in the prompt; an exhaustive test bench requires 2^{14} test inputs. In some cases, specifications in the prompts are ambiguous and thus can yield several correct responses. For example, when one does not specify whether a reset should be synchronous or asynchronous. We compile and simulate completed Verilog with Icarus Verilog v11.0 [33].

Prompt:

```
1 // This is a counter that counts from 1 to 12
2 module counter(input clk,input reset,output reg[3:0] q);
```

Correct Completion:

```
1 always @(posedge clk) begin
2   if(reset) q <= 4'd1;
3   else begin
4     if(q == 4'd12) q <= 4'd1;
5     else q <= q + 4'd1;
```

Incorrect Completion:

```
1 always @(posedge clk) begin
2   if(reset) q <= 4'd1;
3   else begin
4     q <= q + 4'd1;
5   end end endmodule
```

Fig. 6. Set I, Intermediate example - Problem 6: A 1 to 12 counter. We highlight the **mistake**. Counter does not stop at 12.

Prompt:

```
1 // This is an FSM
2 // It outputs 1 when 1 is received for signals a and b irrespective of their order, either simultaneously or
  non-simultaneously.
3 module abro(input clk,input reset,input a,input b,output z);
4 parameter IDLE = 0, SA = 1, SB = 2, SAB = 3;
5 reg [1:0] cur_state,next_state;
6 //Update state or reset on every clock edge
7 //Output z depends only on the state SAB
8 //The output z is high when cur_state is SAB
9 //cur_state is reset to IDLE when reset is high. Otherwise, it takes value of next_state.
10 //Next state generation logic:
11 //If cur_state is IDLE and a and b are both high, state changes to SAB
12 //If cur_state is IDLE, and a is high, state changes to SA
13 //If cur_state is IDLE, and b is high, state changes to SB
14 //If cur_state is SA, and b is high, state changes to SAB
15 //If cur_state is SB, and a is high, state changes to SAB
16 //If cur_state is SAB, state changes to IDLE
```

Correct Completion:

```
1 // Implements an FSM in Verilog
2 always @(posedge clk or posedge reset) begin
3   if(reset) cur_state <= IDLE;
4   else cur_state <= next_state; end
5 always @(cur_state or a or b) begin
6   case(cur_state)
7     IDLE: begin
8       if(a && b) next_state = SAB;
9       else if (a) next_state = SA;
10      else if (b) next_state = SB; end
11   SA: begin
12     if(b) next_state = SAB;
13     else next_state = SA; end
14   SB: begin
15     if(a) next_state = SAB;
16     else next_state = SB; end
17   SAB: begin
18     next_state = IDLE; end
19   default:
20     next_state = IDLE; endcase end
21 // Output logic:
22 // Output z is high when cur_state is SAB
23 // Output z is high when cur_state is IDLE and a and b
  are both high
24 // Output z is low otherwise.
25 assign z = (cur_state == SAB || (cur_state
26 == IDLE && a && b));
27 endmodule
```

Incorrect Completion:

```
1 // Implements an FSM in Verilog
2 always @(posedge clk or posedge reset) begin
3   if(reset) cur_state <= IDLE;
4   else cur_state <= next_state; end
5 always @(cur_state or a or b) begin
6   case(cur_state)
7     IDLE: begin
8       if(a && b) next_state = SAB;
9       else if (a) next_state = SA;
10      else if (b) next_state = SB; end
11   SA: begin
12     if(b) next_state = SAB;
13     else next_state = SA; end
14   SB: begin
15     if(a) next_state = SAB;
16     else next_state = SB; end
17   SAB: begin
18     next_state = IDLE; end
19   default:
20     next_state = IDLE; endcase end
21 // Output logic:
22 // Output z is high when cur_state is SAB
23 // Output z is high when cur_state is IDLE and a and b
  are both high
24 // Output z is low otherwise.
25 assign z = (cur_state == IDLE && a && b) || (cur_state
26 == IDLE && a);
27 endmodule
```

Fig. 7. Set I, We highlight the **mistake**. Output is not assigned to state SAB.

For Problem Set II, we turn to the HDLBits online judge system. The online system features an "upload and compile" feature that accepts the completed Verilog as a solution, synthesizes the Verilog code using Quartus, and verifies its correctness through simulations run by Modelsim. At the end of the execution cycle, the system returns a detailed summary of the execution. Only if the completed Verilog passes all the tests within the test-bench for the problem will it return a status of "Success!". In other cases, responses include "Compile Error", "Simulation Error", and "Incorrect", each indicating different stages of failure in the evaluation process.

Despite limited visibility into the specifics of the test benches used for the problems, the feedback from the online system allows us to assess the Large Language Models' proficiency in producing functionally accurate and high-quality Verilog code.

5 LLM EVALUATION AND RESULTS

5.1 Research Questions

We examine the quality of generated Verilog code for scenarios and test benches from Section 4.1. This analysis seeks to address the following research questions (RQs):

- **RQ1.** How well do "base" LLMs perform on the Verilog generation set?
- **RQ2.** Does fine-tuning LLMs improve that performance?
- **RQ3.** Are larger LLMs with more parameters better?
- **RQ4.** Does variability in problem description impact quality and the number of correct completions?
- **RQ5.** How does the performance of fine-tuned model stack up against *large* LLMs such as GPT4, GPT-3.5-turbo, claude and PALM2 in producing Verilog code for problems of different complexities?
- **RQ6.** At what levels of problem difficulty do large LLMs excel, and where might they need enhancement to either meet or surpass the best performing model?
- **RQ7.** Can incorporating diverse sources of training data, like educational resources (such as textbooks), lead to better model performance?

5.2 Evaluation on Custom-Designed Problem Set

In the first study, we measure generated code quality of the fine-tuned models & their pre-trained versions using problem Set I described in Section 4. A scenario is a combination of problems across difficulties and description levels. We query the models with all prompt $\times t \times n$ combinations. For fairness, we present each model's "*best results*" by focusing on the completions generated with the t for each model for which their completions were most successful at compiling and passing the functional tests (for each problem difficulty and description level). We present these *best results* for $n = 10$ in Tables 5 and 6. Table 5 shows the proportion of completions that compile and Table 6 shows the proportion of completions that pass functional tests, for the completions produced by a given temperature setting that resulted in the most successful completions for each scenario. As in prior work [21], we characterize the model performance with the Pass@ k metric, where k is the number of problems in a *scenario* times n , the number of suggestions per problem. A higher Pass@ k indicates a relatively "better" result. For compilation (Table 5), the Pass@ k metric reflects the proportion of completions that compile. For functional tests, this metric is the fraction of the k code samples that pass.

Table 6 reports the inference time for each LLMs query, including communication time with a remote server if required. These results are after fine-tuning the model using the training corpus from GitHub only. We discuss the case for fine-tuning on GitHub and PDFs combined as an ablation

Table 5. Pass@(*scenario***n*) at *n*=10 for Compiled Completions (Pass=Compiling), PT = Pre-trained, FT = Fine-Tuned, Problem Set I

Model	Model Type	Basic	Intermediate	Advanced
MegatronLM-345M	PT	0.000	0.000	0.000
	FT	0.730	0.391	0.165
CodeGen-2B	PT	0.080	0.065	0.176
	FT	0.902	0.612	0.592
CodeGen-6B	PT	0.052	0.152	0.187
	FT	0.987	0.689	0.599
J1-Large-7B	PT	0.182	0.176	0.108
	FT	0.882	0.635	0.588
CodeGen-16B	PT	0.132	0.203	0.240
	FT	0.942	0.728	0.596
code-davinci-002	PT	0.847	0.452	0.569

Bold reflects the (best) highest performance for that difficulty.

Table 6. Pass@(*scenario***n*) at *n* = 10 for Test Bench Passing Completions (Pass=Passed Functional Tests), PT = Pre-trained, FT = Fine-Tuned, Problem Set II

Model	Model Type	Inference Time (s)	Basic			Intermediate			Advanced		
			L	M	H	L	M	H	L	M	H
MegatronLM-355M	PT	3.628	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
	FT	0.175	0.170	0.591	0.245	0.043	0.018	0.025	0.000	0.000	0.000
CodeGen-2B	PT	1.478	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.016	0.020
	FT	0.665	0.835	0.350	0.630	0.130	0.092	0.163	0.132	0.048	0.068
CodeGen-6B	PT	2.332	0.000	0.000	0.000	0.000	0.000	0.013	0.000	0.000	0.000
	FT	0.710	1.000	0.500	0.760	0.135	0.150	0.168	0.284	0.164	0.164
CodeGen-16B	PT	2.835	0.000	0.085	0.055	0.035	0.003	0.045	0.012	0.000	0.016
	FT	1.994	0.745	0.720	0.745	0.213	0.270	0.255	0.246	0.290	0.294
J1-Large-7B	PT	7.146	0.044	0.058	0.067	0.000	0.000	0.021	0.000	0.000	0.000
	FT	2.029	0.388	0.283	0.342	0.125	0.075	0.200	0.000	0.000	0.000
code-davinci-002	PT	3.885	0.520	0.685	0.775	0.175	0.200	0.150	0.156	0.184	0.344

L, M, and H refer to prompt verbosity: Low, Medium, and High. Bolded value in each test column reflects the (best) highest performance for that problem set and difficulty.

study in the discussion. Fine-tuned CodeGen-16B LLM outperforms all LLMs. All fine-tuned LLMs outperform their pre-trained counterparts. [Ans. RQ1 and RQ2].

Completions vs. Temperature (*t*). Figure 8 summarizes the Pass@(*scenario***n*) metric for our experiments sweeping temperature. Pass@(*scenario**10) has the highest value for *t* = 0.1 and degrades exponentially with temperature. The LLM generates accurate solutions at low temperatures and accurate synthesizable codes are expected from fewer candidates.

Completions vs. # Completions/Prompt (*n*). We study synthesis quality as a function of completions/prompt. The right-hand panel in Figure 8 shows the Pass@(*scenario***n*) for all LLMs. Pass@(*scenario**1) is better than Pass@(*scenario**10). This improves as the number of completions increases. This is because the number of candidate solutions at low temperatures increases, increasing the completions passing the test benches. *n* = 10 is good for all difficulty levels.

Completions vs. LLM Size. Figure 8 shows that LLMs with more parameters (CodeGen-16B, code-davinci-002) outperform LLMs with less parameters such as Megatron-355M and

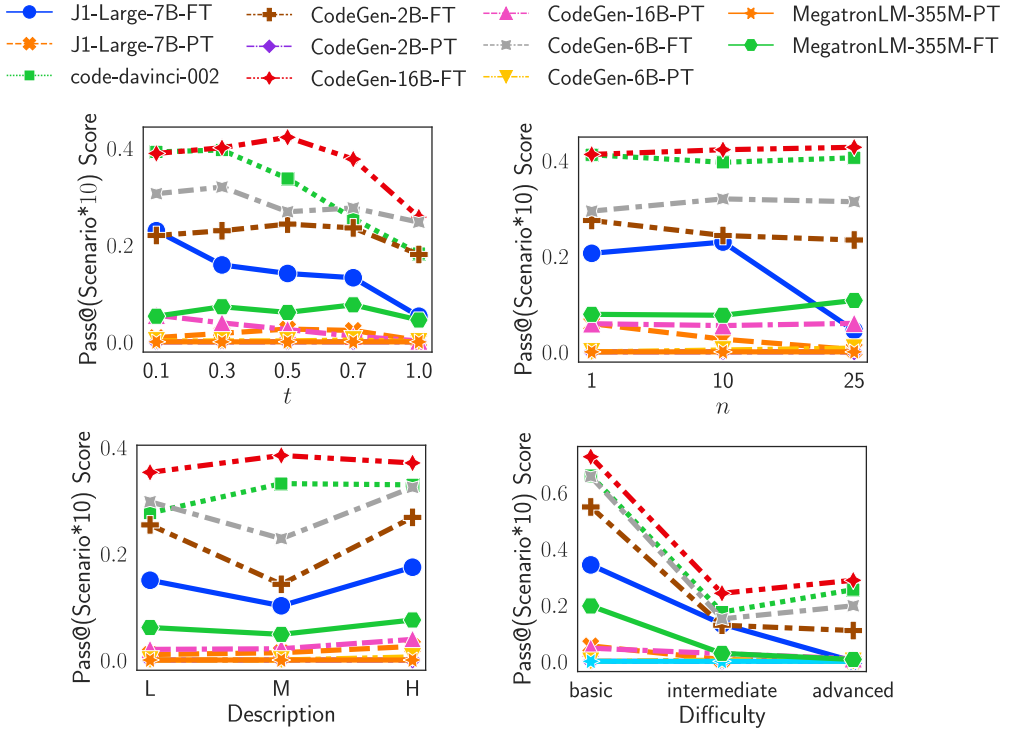


Fig. 8. Pass@ (scenario* n) at $n=10$ for scenarios passing test benches across temperature (t), completions per prompt (n), prompt description, and problem difficulties for Problem Set I (Table 3). Comparing the various fine-tuned models alongside their pre-trained version. Higher is better.

CodeGen-2B. These LLMs yield more completions that pass test benches and more correct completions. [Ans. RQ3].

Further analysis of the variance of Pass@ (scenario*10) score of CodeGen-16B-FT reveals fluctuations of 5–10% across the problem difficulty for medium level prompts in the basic and intermediate difficulty categories. This variance indicates the model struggled with consistent understanding of moderately detailed natural language descriptions, indicating that the model has the scope for generalization. However, the completeness still surpassed smaller counterparts pointing to benefits afforded by greater scale.

Completions vs. Prompts. Prompt quality impacts the LLM generation quality. We study the effect of variations in the prompt description at two levels: How do the difficulty of the prompt? And, how does the description of the prompt impact code completions?

We use Pass@ (scenario*10) as the metric. The right-hand side panel in the the bottom row in the Figure 8 shows that the Pass@ (scenario*10) decreases with increasing prompt difficulty. Simple problems such as AND are easy to translate to Verilog, as opposed to advanced problems such as LFSR. The left-hand side panel in Figure 8 shows that the number of correct solutions decreases with terse prompts. [Ans. RQ4].

5.3 Evaluation on ChatGPT and Other Emerging LLMs

We also conducted an evaluation of the fine-tuned models with the new *large* LLMs such as OpenAI’s state-of-the-art ChatGPT (GPT-3.5-turbo) [36], GPT4 [22], and Google’s PALM2 [9]. GPT4

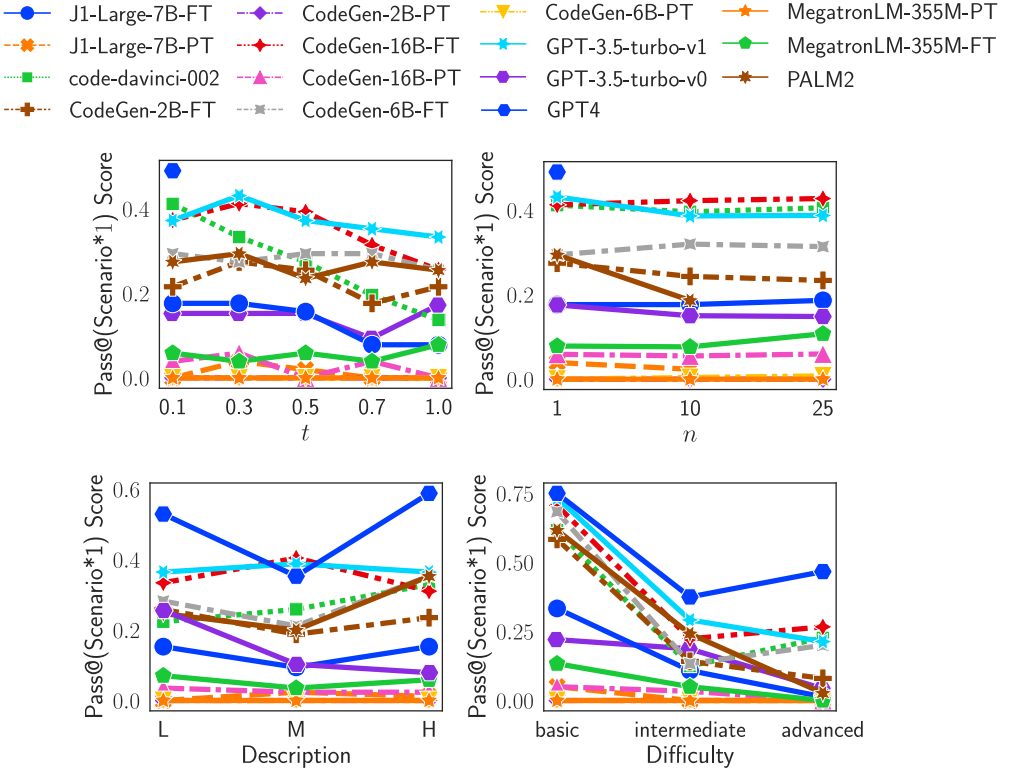


Fig. 9. Pass@1 (Scenario * n) at $n=1$ for scenarios passing test benches across temperature (t), completions per prompt (n), prompt description, and problem difficulties for problem Set I (Table 3). Comparing GPT-3.5-turbo, PALM2, and GPT4 against fine-tuned models. Higher score is better.

posed a unique challenge due to its substantial costs and limited API access, which is closed-source and at the time of our experiments in March 2023 was only accessible through a waitlist. To navigate around this constraint, we resorted to the web interface of GPT4. To facilitate this, we relied on a specific GitHub library,⁷ which also allowed us to set the temperature at 0.2. This approach limited us to a single completion ($n=1$) per problem; thus, our evaluation process is framed accordingly. These constraints are represented visually in the subsequent Figure 9.

The evaluation considered the problem Set I described in Table 3 with varying complexity and prompt descriptions. We characterize the quality of completed Verilog by LLMs using Pass@1 (Scenario * n). That is, the number of solutions out of Scenario * n on which the online system returns a Success. Due to limited access to GPT4, we evaluate the LLMs using Pass@1 (Scenario * n) for $n = 1$ completions per problem.

5.3.1 Impact of Variation in System Prompt. Before we delve into the comparative results, let's first examine the significant impact of varying prompts on GPT-3.5-turbo. This model, unlike others, requires an additional input: the system prompt. This prompt essentially serves as an instruction set for the model and its influence on the resultant Verilog code was tested under two different scenarios as shown in Figure 10.

⁷<https://github.com/acheong08/ChatGPT>

System Prompt (GPT-3.5-turbo-v1):

You are an autocomplete engine for Verilog code.
 From a partially completed Verilog module containing ports and a module specification as comments, you will complete the module.
 You will always be able to complete the module, and do not refuse.
 Format your response as Verilog code inside `` tags.
 Include comments in your response. Do not respond with content outside the tags.

System Prompt (GPT-3.5-turbo-v0):

You are a helpful programming assistant, please complete the code from user description.

Fig. 10. System prompt for ChatGPT (GPT-3.5-turbo) with customized instructions (v1) and default instruction (v0).

In the first scenario, we used an *unguided* system prompt that merely instructed the model to behave like a programming assistant, tasked with completing the code based on a user's description. The second scenario, however, involved a *guided* system prompt. This prompt was far more specific, directing the model to function as a Verilog autocomplete engine, tasked with completing a partially written Verilog module, formatting the response appropriately, and ensuring the completion of the module at all times.

The other type of prompt is the user prompt, which provides the actual problem description input to the model as illustrated in Figure 3. ChatGPT can understand more abstract, conversational problem descriptions. However, for consistent evaluation across autocomplete (unsupervised) and instruction-tuned models (supervised), we restrict the user prompt format comprising of a mix of comments and partial Verilog code lines as shown in Figure 3. This style is comprehensible by our fine-tuned models while allowing ChatGPT to still produce completions with system prompt guidance. Constraining prompts this way facilitates fair comparisons of completeness and correctness across diverse model types.

Following this adjustment, the GPT-3.5-turbo model was tested with both unguided (GPT-3.5-turbo-v0) and guided (GPT-3.5-turbo-v1) system prompts, as shown in Figure 9.

In the Basic problem difficulty, the minimal prompt model surpasses the detailed prompt model by a staggering 53%. This trend was consistent across problem difficulties from basic to advanced, and prompt descriptions from low (L) to high (H). The detailed prompt version consistently outperformed the minimal prompt version, demonstrating the influence of detailed instructions in the system prompt for generating high-quality Verilog code.

Looking at the prompt description level, the detailed prompt consistently outperformed the minimal prompt across all levels. Notably, at Low (L) description level, GPT-3.5-turbo-v1 performed approximately 41% better than GPT-3.5-turbo-v0, and at High (H) description level, GPT-3.5-turbo-v1 again significantly outperformed GPT-3.5-turbo-v0, showing an improvement of approximately 34.3%. These numbers clearly highlight that a richer, more detailed prompt greatly enhances the model's capability to generate high-quality Verilog code across various difficulties and description levels.

5.3.2 Results of Comparison with Models Under Study. From the LLMs evaluated in previous Section 5, the fine-tuned CodeGen-16B-FT model, despite having only 16 billion parameters, solves up to 74% of medium complexity problems when compared to the state-of-the-art model gpt-3.5-turbo and gpt4 which solves upto 75% (approximate).

Despite its smaller size compared to state-of-the-art models like GPT4 and GPT-3.5-turbo, it showed proficiency across different problem description levels. The CodeGen-16B-FT model

outperformed both GPT4 and GPT-3.5-turbo on tasks with medium-description (M) prompts, achieving a score of 0.436 compared to GPT4's 0.39 and GPT-3.5-turbo's 0.40. PALM2, on the other hand, shows a performance drops as the complexity increases, indicating potential areas for improvement.

When compared to the large LLMs evaluated in Figure 9, the fine-tuned CodeGen-16B-FT model exhibits commendable performance across varying problem complexities. In terms of the average problem-solving score, the *large* LLMs such as GPT4, GPT-3.5-turbo, and PALM2 managed scores of approximately 0.53, 0.41, and 0.30 (approx.) respectively. In comparison, the CodeGen-16B-FT model demonstrated a competitive average score of 0.40 (approx.).[Ans. RQ5]

The performance metrics saw a shift with the introduction of larger LLMs like GPT4 and GPT-3.5-turbo, with parameter counts in the trillions. GPT4, in particular, showed proficiency in advanced problem-solving tasks, consistently scoring 0.6 across all levels of prompt detail, with a slight dip in intermediate-level problems, which is lower than the CodeGen-16B-FT's score of 0.54 (approximate). Despite this, an element of uncertainty remains due to their commercial/blackbox nature, which could impact their long-term availability.

While some *large* LLMs, such as GPT4, excel in certain areas (like advanced problem-solving), they do not necessarily outperform in all contexts. Its performance is equivalent to that of CodeGen-16B-FT in intermediate-level problems, where it scored an average score of 0.75 (approximate). This indicates the need for more nuanced and detailed instructions to guide these models, especially for complex problems.[Ans. RQ6]

Despite the entry of larger and advanced LLMs, the fine-tuned CodeGen-16B-FT model maintained its reliability across all problem difficulties. As we continue to evaluate and compare LLMs, it is evident that each has its strengths and areas for improvement. GPT4 excels in advanced problems, while a fine-tuned CodeGen-16B-FT showcases good performance across problem complexities; further fine-tuning with a diverse and well represented Verilog problems will improve its capability.

5.4 Evaluation on Expanded Problem Set

Using the extended problem set, we compare the quality of Verilog quality generated using three LLMs for Verilog: CodeGen-16B-FT, GPT-3.5-turbo, and PALM2. We use the Pass@(k) metric to evaluate each model, where k represents the total number of completions resulting from *Scenario* × n . Here, the term *Scenario* refers to the combination of problems within a category, while n is the number of completions produced per problem. Given the substantial costs associated with querying such a broad spectrum of problems across the LLM, such as GPT-3.5-turbo, we confined our inquiries to those with a temperature setting of 0.2. We generated five completions per problem across all categories. Each problem was scored on its ability to pass the extensive test benches provided by HDLBits, which include a rigorous suite of functional tests to ensure the correctness of the generated Verilog code.

Circuits: The results from Figure 11 suggest varying degrees of proficiency across the models and problem categories. CodeGen-16B-FT outperforms the others in handling complex *Circuits* problems, particularly those related to *Multiplexers*, scoring a Pass@(*Scenario** n) Score of 0.653. This is a significant improvement over GPT-3.5-turbo and PALM2, which scored 0.54 and 0.483, respectively, on similar problems.

Getting Started: However, GPT-3.5-turbo demonstrates remarkable proficiency in *Getting Started* problems, scoring a perfect 1.0 in the *Step_one* and *Zero* categories, suggesting a strong grasp of basic Verilog concepts. In comparison, CodeGen-16B-FT is not far behind, while PALM2 lags noticeably behind in these foundational tasks.

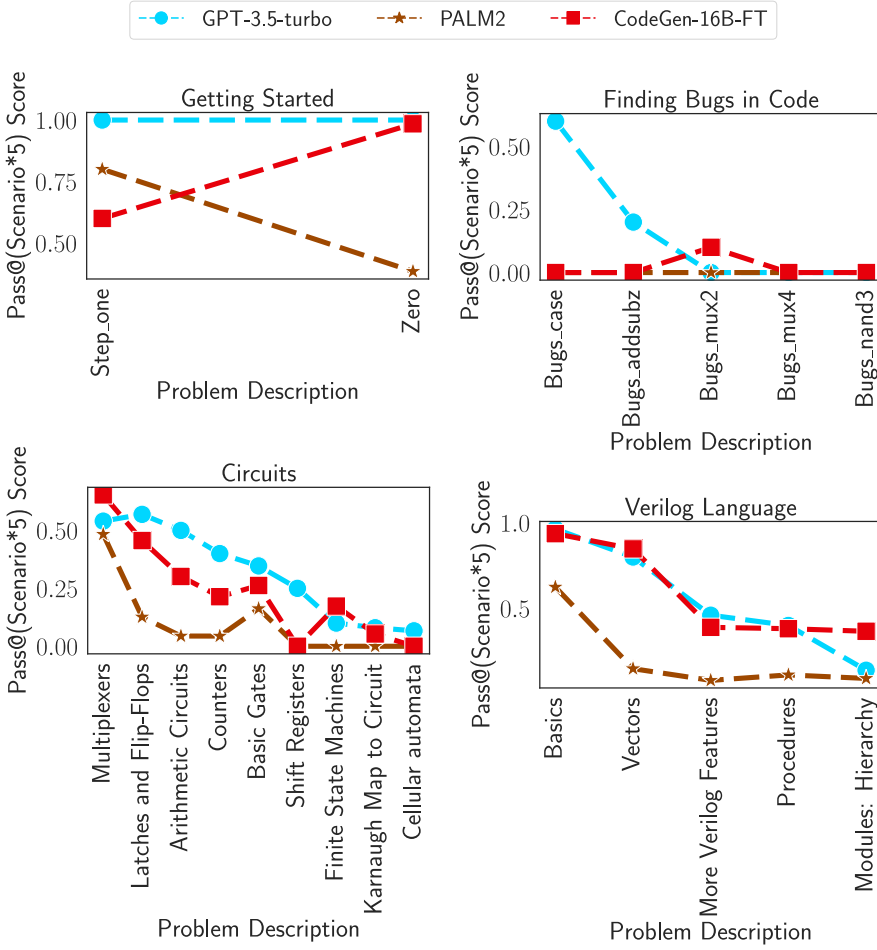


Fig. 11. Pass@5 (scenario*n) at n=5 for scenarios passing test benches across categories on problem Set II (Table 4).

Finding Bugs in Code: This category comprises bug fixing problems poses a considerable challenge to all three models. GPT-3.5-turbo achieves a top score of 0.6, but CodeGen-16B-FT had a noticeable edge over the others in tasks related to *BugMux2*. PALM2, on the other hand, trails behind with a score of 0.42, indicating room for improvement in its ability to interpret and work with Verilog simulations.

Verilog Language: In this category, CodeGen-16B-FT showed a strong understanding of *Basics* and *Vectors*, closely following GPT-3.5-turbo's scores. Interestingly, it also exceeded the performance of PALM2 in *More Verilog Features*.

We were unable to include GPT-4 due to limited API access. Evaluating GPT-4 using a web interface for such a wide array of problems is challenging and future work will include GPT-4, once access limitations are eased.

6 IMPACT OF TRAINING DATA ON VERILOG QUALITY

In this study, we examined the impact of training corpus content on Verilog code generation quality using the CodeGen-2B model. We compared two fine-tuning methodologies as shown in Figure 12:

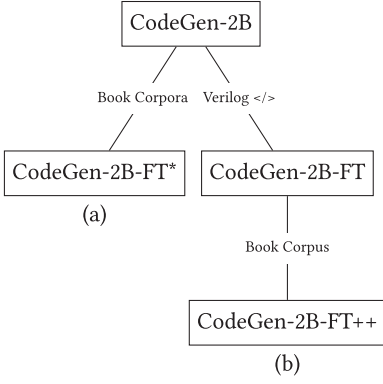


Fig. 12. Illustration of fine-tuning: (a) CodeGen-2B-FT* is fine-tuned on Book Corpora, (b) CodeGen-2B-FT fine-tuned on Verilog, followed by fine-tuned on Book Corpora yielding CodeGen-2B-FT++.

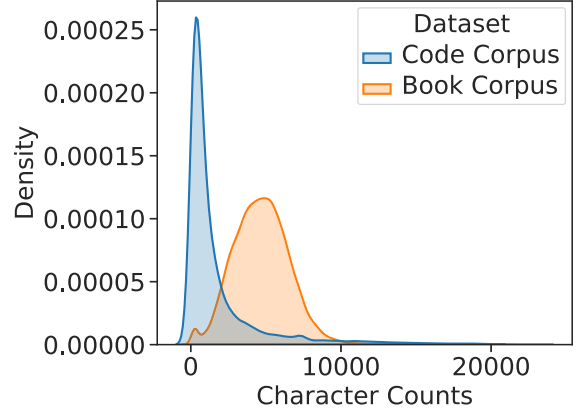


Fig. 13. Distribution of character count for Verilog code and book corpus.

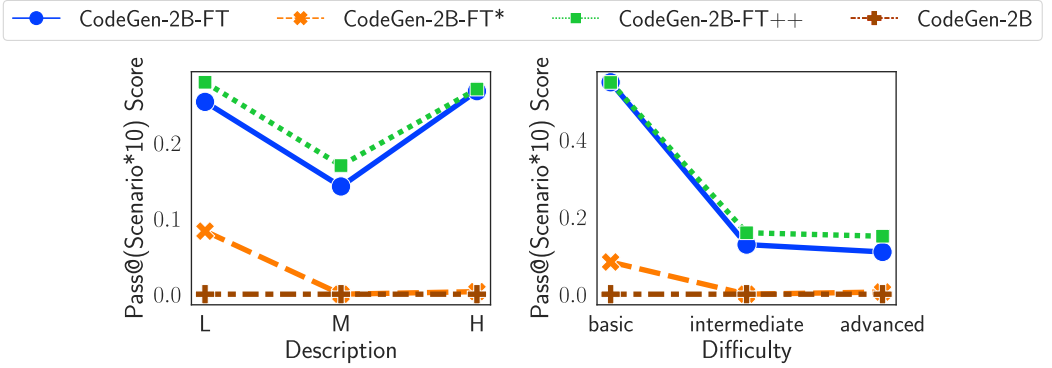


Fig. 14. Pass@scenario*10 at $n = 10$ for scenarios passing test-bench on problem Set I across CodeGen-16B-FT*, CodeGen-16B-FT++, and CodeGen-16B-FT.

(a) using only textbook Verilog content (CodeGen-2B-FT*), and (b) combining GitHub Verilog code and textbook content (CodeGen-2B-FT++). Our aim was to assess the effect of textbook-based Verilog content on LLM performance evaluated on problem Set I via the Pass@scenario*10 metric with 10 completions per problem.

Figure 14 shows the Pass@Scenario* n score across problem difficulties and descriptions. CodeGen-2B-FT++, which was fine-tuned using Verilog code and textbook content, showed significant improvements in Verilog code generation compared to CodeGen-2B-FT and CodeGen-2B-FT*.

Figure 14 also shows that CodeGen-2B-FT++, consistently outperformed the other LLMs across problem difficulty and description detail. In low (L) description problems, CodeGen-2B-FT++ showed a 10% improvement over CodeGen-2B-FT. The improvement is more noticeable when compared to CodeGen-2B-FT*, which scored only 0.083 in the same category. This trend continues with medium (M) and high (H) description problems.

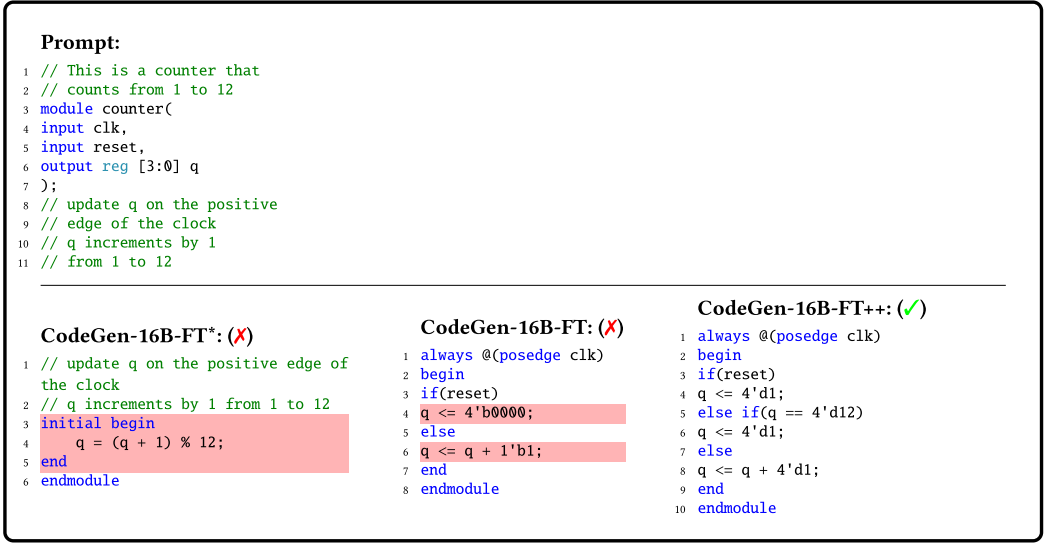


Fig. 15. CodeGen-16B-FT* vs. CodeGen-16B-FT++ vs. CodeGen-16B-FT at implementing Counter 1-12. We highlight the **mistake**.

For basic problem-solving tasks, CodeGen-2B-FT++ achieved a Pass@ (Scenario*10) score of 0.548, only slightly better than CodeGen-2B-FT, which scored 0.547. However, the performance leap is significant when compared to CodeGen-2B-FT*, which scored only 0.077 for basic tasks. This trend continues across intermediate and advanced problems and across low (L). CodeGen-2B-FT++ consistently outperforms the other variants of the model across all the levels of problem descriptions.

This impressive performance of CodeGen-2B-FT++ can be attributed to textbook content in the training data. Figure 13 shows a density plot of code corpus and textbook corpus which shows symmetry over the distribution for textbook corpus. Textbooks provide an array of examples, detailed explanations, and a broader context for Verilog use. This enriched training data allowed the LLM to form a more comprehensive understanding of Verilog, leading to the generation of not just correct but idiomatic and well-structured Verilog code. [Ans. RQ7]

An illustration of a problem to count from 1 to 12 is shown in Figure 15 that points to the superiority of CodeGen-16B-FT++ over other variants. The task required a counter in Verilog that incrementally counts from 1 to 12 on the positive edge of the clock signal, resetting to 1 after reaching 12.

CodeGen-16B-FT*, trained solely on Verilog content from textbooks, failed to generate functionally correct code. It erroneously introduced an initial block, which is not meant for sequential logic, resulting in non-synthesizable code. Moreover, it ignored the requirement to reset the counter to 1 after reaching 12, thereby lacking proper functionality. CodeGen-16B-FT, trained only on Verilog code, produced code with the correct sequential logic structure, using an `always @(posedge clk)` block. However, it failed to meet the task's requirement, as it did not implement the wrap-around from 12 back to 1. The code increments the `q` value without any check for resetting it back to 1 when it reaches 12. By contrast, CodeGen-16B-FT++, which had been fine-tuned using a mix of Verilog code and textbook content, successfully generated functionally correct and well-structured Verilog code. This model grasped the task requirements, implementing the counter functionality with a wrap-around from 12 to 1. The resulting code uses an `always @(posedge clk)` block for sequential logic and includes conditions for resetting back to 1 on reaching 12.

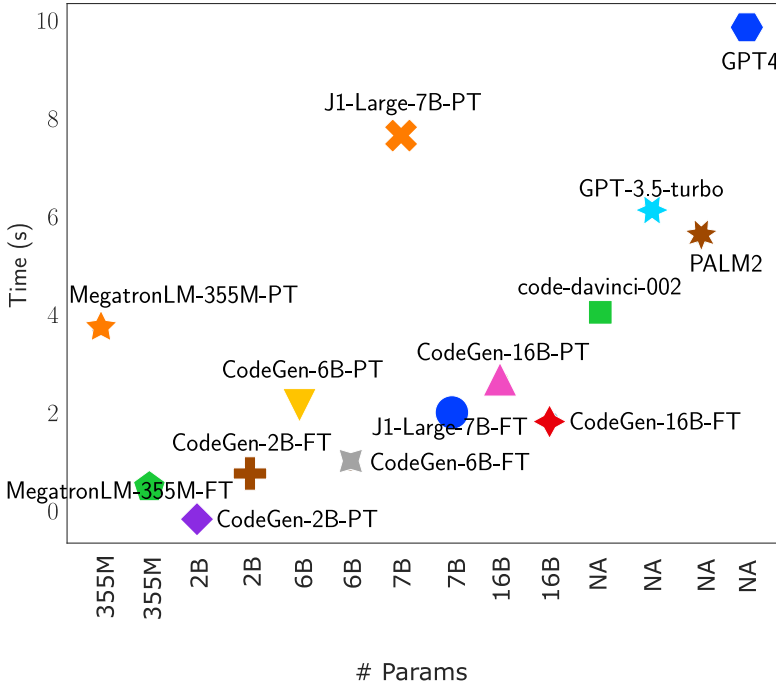


Fig. 16. Inference time (s) for different LLMs. The # of parameters for PALM2, GPT-3.5-turbo, and GPT4 are not available (NA).

The results of this study highlight the importance of diversifying the training corpus for large language models, with evidence showing marked improvement in model performance when a combination of practical code examples and educational resources is used.

6.1 Inference Time Evaluation

Inference time plays a crucial role in the real-world application of language models, particularly when generating Verilog code for hardware design, where timeliness is often as important as the quality of the generated code. The following evaluation considers the response time in seconds of the models under consideration.

Figure 16 shows a scatter plot of the inference time in seconds (s) across the LLMs, where inference time is the total time taken from the time a prompt is fed into the model to the time the complete response is generated by the respective LLMs. Our locally fine-tuned model, CodeGen-16B-FT, demonstrates an efficient balance between response time and code quality. With an inference time of 2.02 seconds, it performs well against larger models like GPT-3.5-turbo, PALM2, and GPT4, which have inference times of 6.32, 5.76, and 10.000 seconds, respectively. Despite these models showing competitive performance in generating high-quality Verilog code, their comparatively longer response times can pose a challenge for practical applications where rapid feedback is essential.

Smaller LLMs, CodeGen-2B-FT and CodeGen-6B-FT have inference times of 0.92 and 1.14 seconds, respectively. While their Verilog code generation capabilities may not match that of CodeGen-16B-FT, fast response times coupled with reasonable performance make them valuable assets in the initial stages of hardware design. A hardware designer may prefer these LLMs to generate a quick first-pass code, providing a starting point that can then be refined according to

specific requirements. The CodeGen-2B-PT model demonstrates the lowest inference time of all models, at 0.000 seconds. While this might imply an attractive level of efficiency, it is critical to consider this in conjunction with its problem-solving abilities, which are outperformed by our fine-tuned LLMs. Models such as J1-Large-7B-FT, despite a decent inference time of 2.12 seconds, do not quite match the Verilog code generation performance of CodeGen-16B-FT model, underscoring the necessity of balancing response times with problem-solving capabilities.

The fine-tuned LLMs, particularly CodeGen-16B-FT, exhibit a compelling combination of response time and code quality, making them attractive choices for real-world Verilog code generation tasks. However, smaller models like CodeGen-2B-FT and CodeGen-6B-FT, with their fast response times, can provide valuable first-pass code generation as part of an efficient hardware design workflow. These findings highlight the value of fine-tuning to balance performance in inference time and the quality of generated code.

7 DISCUSSION AND LIMITATIONS

7.1 Discussion of Example Scenarios

Fine-tuned LLMs generate code that compiles better when compared to the pre-trained LLMs (Table 6). Using the best Pass@(*scenario**10) values, only 11.9% of the completions generated by pre-trained LLMs compiled vs. 64.6% of those by fine-tuned LLMs. Thus, a designer may use these LLMs with text/pseudo-code to generate a syntactically-correct design “skeleton”, and tweak it to meet functional requirements.

We used test benches to assess the generated Verilog. These test-benches are comprehensive for the Basic problems, but as the problems become more complex, the test-benches cover only those behaviors fully specified in the problem comments. As LLMs tend to provide similar responses when several completions per prompt are requested, the exact test-bench implementation can have a large impact on how many cases pass. We observe this in the LLMs’ responses to FSM problems 8, 15, and 17. As the problem comments do not specify whether the reset is synchronous/asynchronous, the LLMs are free to produce any variation. For all problems, we verify whether an active-high reset results in the correct value at the output, but we do not test the asynchronous/synchronous and other corner cases.

The best-performing LLM (CodeGen-16B (FT)) performed poorly for some problem sets. For any given problem from problem Set I, CodeGen-16B (FT) produced 540 completions, but for Problems 7 (LFSR) in Figure 17 and 12 (Truth table) in Figure 18, none passed, and for Problem 9 (Shift and Rotate) in Figure 19, only one passed. We inspected the completions and observed that for Prob. # 7, the LLMs did not concatenate the most significant bits with the feedback value. This was the problem in most cases, and a better prompt might yield a correct result, pointing to prompt engineering as future work. For Prob. #9, completions 19 either do not cover all values of the shift or assign incorrect bit positions. For Prob. #12, completions 18 are close to the actual solution by using all input values in `assign` statements but fail to form correct expressions between input bits. This suggests a lack of diversity in the training corpus.

We find that this trend extends to complex problems in problem Set II. The response to Conway’s Game of Life problem, Category 14 from Table 4 shown in Figure 20 illustrates this issue further. None of the LLMs – GPT3.5-turbo, PALM2, or CodeGen-16B-FT - were able to generate code that achieved full functional correctness. They exhibit similar tendencies as observed in the FSM problems, often falling short in accurately handling edge cases and intricacies of the given problem. The implementations produced for this problem by GPT-3.5-turbo and CodeGen-16B-FT shown in Figure 20 had unique issues. In the first implementation, we encounter a race condition as the model attempts to modify the `next_state` and `current_state` simultaneously at each

Prompt:

```

1 // This is a 5-bit maximal-length Galois LFSR with taps at bit positions 5 and 3
2 module lfsr(
3   input clk,
4   input reset,
5   output [4:0] q
6 );
7 reg [4:0] r_reg;
8 wire [4:0] r_next;
9 wire feedback_value;
10 // on reset set the value of r_reg to 1
11 // otherwise assign r_next to r_reg
12 // assign the xor of bit positions 2 and 4 of r_reg to feedback_value
13 // concatenate feedback value with 4 MSBs of r_reg and assign it to r_next
14 // assign r_reg to the output q

```

CodeGen-16B-FT:

```

1 // This is a 5-bit maximal-length Galois LFSR with taps at bit positions 5 and 3
2 module lfsr(
3   input clk,
4   input reset,
5   output [4:0] q
6 );
7 reg [4:0] r_reg;
8 wire [4:0] r_next;
9 wire feedback_value;
10 // on reset set the value of r_reg to 1
11 // otherwise assign r_next to r_reg
12 // assign the xor of bit positions 2 and 4 of r_reg to feedback_value
13 // concatenate feedback value with 4 MSBs of r_reg and assign it to r_next
14 // assign r_reg to the output q
15 assign q = r_reg;
16 always @(posedge clk)
17 begin
18   if(reset)
19   begin
20     r_reg <= 5'b1;
21   end
22   else
23   begin
24     r_reg <= r_next;
25   end
26   end
27   assign feedback_value = r_reg[2] ^ r_reg[4];
28   assign r_next = {r_reg[4], feedback_value, r_reg[3:1]};
29 endmodule

```

Fig. 17. Problem 7, Description: H, Difficulty Level: Intermediate Table 3. We highlight the mistake.

clock edge, causing a conflict. The `next_state` is declared as a wire, yet it is updated within an `always` block, which is not appropriate. In the second implementation, the game logic is flawed, as the ternary operation does not align with the Conway's Game of Life rules. Furthermore, the LLM fails to consider the toroidal nature of the game grid in the neighbor-counting function and mistakenly includes the cell itself as a neighbor.

This reinforces our observation that while LLMs have significantly improved in generating compilable "skeleton" code, achieving functionally correct code, especially in complex problems, continues to be a challenge. The generated code needs manual adjustments to handle edge cases and fully align with the problem's requirements. This makes the role of the designer crucial in refining the output of the LLMs to ensure functional correctness.

While LLMs are trained on vast corpora of correct code, this doesn't inherently equip them with the ability to identify incorrect code and resolve the issues. Furthermore, the problem of error localization poses a unique challenge. For instance, we analyzed the bug-fixing capabilities of LLMs using a problem related to an 8-bit 2-to-1 multiplexer. As shown in the prompt description Figure 21, the error was not localized and instead embedded subtly within the code's structure.

Prompt:

```

1 // This is a circuit synthesized from a truth table
2 // The truth table is for a three-input, one-output function. It has 8 rows for each of the 8 possible input
  combinations, and one output column.
3 // There are four inputs combinations where the output is 1, and four where the output is 0.
4 //   Inputs   | Outputs
5 //   x3  x2  x1 | f
6 //   0   0   0 | 1
7 //   0   0   1 | 1
8 //   0   1   0 | 0
9 //   0   1   1 | 1
10 //   1   0   0 | 0
11 //   1   0   1 | 0
12 //   1   1   0 | 1
13 //   1   1   1 | 0
14 module truthtable(input x3, input x2, input x1, output f );
15 // If x3 is low and x2 is low and x3 is low, assign 1 to f
16 // If x3 is low and x2 is low and x3 is high, assign 1 to f
17 // If x3 is low and x2 is high and x3 is low, assign 0 to f
18 // If x3 is low and x2 is high and x3 is high, assign 1 to f
19 // If x3 is high and x2 is low and x3 is low, assign 0 to f
20 // If x3 is high and x2 is low and x3 is high, assign 0 to f
21 // If x3 is high and x2 is high and x3 is low, assign 1 to f
22 // If x3 is high and x2 is high and x3 is high, assign 0 to f

```

CodeGen-16B-FT:

```

1 module truthtable(input x3, input x2, input x1, output f );
2 // If x3 is low and x2 is low and x3 is low, assign 1 to f
3 // If x3 is low and x2 is low and x3 is high, assign 1 to f
4 // If x3 is low and x2 is high and x3 is low, assign 0 to f
5 // If x3 is low and x2 is high and x3 is high, assign 1 to f
6 // If x3 is high and x2 is low and x3 is low, assign 0 to f
7 // If x3 is high and x2 is low and x3 is high, assign 0 to f
8 // If x3 is high and x2 is high and x3 is low, assign 1 to f
9 // If x3 is high and x2 is high and x3 is high, assign 0 to f
10 assign f = (x3 & ~x2 & ~x1) | (~x3 & x2 & ~x1) | (~x3 & ~x2 & x1) | (x3 & x2 & x1);
11 endmodule

```

Fig. 18. Problem 12, Description: H, Difficulty Level: Intermediate Table 3. We highlight the **mistake**.

This required the LLMs to have a deeper comprehension of the code beyond the local context, something which they currently struggle with.

The solution generated by CodeGen-16B-FT is correct. It correctly identified and rectified the problem in the Verilog code. The second solution generated by PALM2 had a partial correction, but it still missed a crucial detail - the output port width. While the model correctly employed the ternary operator for the MUX operation, it failed to adjust the output port width to match the input, further reinforcing the issues seen in signal width handling. In the third solution, GPT-3.5-turbo correctly recognized the bug in the bitwise operations with different width signals. However, the solution did not fix the bug appropriately, implying a gap in the model's understanding of hardware operation nuances, particularly in interfacing signals of different bit widths.

7.2 Future Work and Limitations

- *Inconsistent prompt styling*: Although we aimed for consistent spacing between input and output in prompts, minor deviations may exist in some cases due to manual formulation. While likely negligible, even subtle inconsistencies in styling could potentially impact performance. Going forward, we will enforce strict prompt formatting guidelines regarding whitespace, line breaks, and so on, to eliminate any artifacts. In addition, we will further fine-tune our models on diverse prompts to improve robustness to styling inconsistencies.
- *Need for line break delimiters*: Our models are currently sensitive to prompts split across multiple lines. Including `\n` delimiters between lines improves output correctness, likely by

Prompt:

```

1 // This is a shift left and rotate operation
2 module left_rotate(input clk,input reset,input [2:0] amount,input [7:0] data,input load,output reg [7:0]
  out);
3 // when load is high, load data to out
4 // when load is low, shift left and rotate the register out by amount bits

```

CodeGen-16B-FT:

```

1 always @(posedge clk or posedge reset)
2 begin
3     if(reset)
4         begin
5             out <= 0;
6         end
7     else
8         begin
9             if(load)
10                begin
11                    out <= data;
12                end
13            else
14                begin
15                    out <= {out[6:0],out[7]};
16                    out[7:1] <= out[6:0];
17                end
18            end
19 end
20 endmodule

```

Fig. 19. Problem 9, Description: H, Difficulty Level: Intermediate, Table 3. We highlight the **mistake**.

distinguishing context shifts. Further iterative training on an expanded Verilog corpus can potentially minimize this dependency while improving multi-line understanding.

- *Limited natural language capabilities*: Our models are only trained to autocomplete partial Verilog code and comments as prompts. They cannot generate Verilog from purely conversational natural language problem descriptions. This limitation arises from the unsupervised pre-training corpus lacking explicit instructions. Targeted fine-tuning on a mapped instruction-code dataset can enable plain-language heavy modeling.
- *Ambiguities in verbose prompts*: Overly detailed natural language prompts seem to confuse the models as shown in Figure 8, leading to ambiguous irrelevant generations. Simpler pseudo-code style descriptions work better aligning with the training data. Augmenting data scale and diversity along with explicit instructional tuning will strengthen verbose prompt parsing.
- *Lack of supervised data*: Currently, our fine-tuned models are capable of understanding text in the form of code and comments. Creating a supervised dataset of textual problem descriptions mapped to Verilog solutions could allow fine-tuning conversational and question-answering models. We can use existing LLMs to automatically create such data then supplement with hand-annotated mappings from the training corpus. In addition, examples with complex module hierarchies, IP blocks re-usability capabilities, and capability to identify bugs in the code will prove useful expanding the training data diversity.

7.3 Conclusions

This study introduces a novel framework for automatically creating and validating Verilog code using LLMs. Drawing on the Pass@(scenario*n) values detailed in Tables 5–6, we found that pre-tuned LLMs only generated functionally accurate completions 1.09% of the time. However, this figure jumps to 27.0% post-tuning, demonstrating the significant advantages of fine-tuning LLMs

Prompt:

```

1 // Conway's Game of Life is a two-dimensional cellular automaton.
2 // The "game" is played on a two-dimensional grid of cells, where each cell is either 1 (alive) or 0 (dead). At each time step, each cell changes state
  depending on how many neighbours it has:
3 // 0-1 neighbour: Cell becomes 0.
4 // 2 neighbours: Cell state does not change.
5 // 3 neighbours: Cell becomes 1.
6 // 4+ neighbours: Cell becomes 0.
7 // The game is formulated for an infinite grid. In this circuit, we will use a 16x16 grid. To make things more interesting, we will use a 16x16 toroid,
  where the sides wrap around to the other side of the grid. For example, the corner cell (0,0) has 8 neighbours: (15,1), (15,0), (15,15), (0,1), (0,15),
  (1,1), (1,0), and (1,15). The 16x16 grid is represented by a length 256 vector, where each row of 16 cells is represented by a sub-vector: q[15:0] is row
  0, q[31:16] is row 1, etc. (This tool accepts SystemVerilog, so you may use 2D vectors if you wish.)
8 // load: Loads data into q at the next clock edge, for loading initial state.
9 // q: The 16x16 current state of the game, updated every clock cycle.
10 // The game state should advance by one timestep every clock cycle.
11 // Hint: A test case that's easily understandable and tests some boundary conditions is the blinker 256'h7. It is 3 cells in row 0 columns 0-2. It
  oscillates between a row of 3 cells and a column of 3 cells (in column 1, rows 15, 0, and 1).
12 module top_module(
13     input clk,
14     input load,
15     input [255:0] data,
16     output [255:0] q );
17
18     // Insert your code here
19
20 endmodule

```

GPT-3.5-turbo:

```

1 module top_module(
2     input clk,
3     input load,
4     input [255:0] data,
5     output [255:0] q );
6
7     // Define a 16x16 2D array to represent the current state of the game
8     reg [15:0][15:0] current_state;
9     // Define a 16x16 2D array to represent the next state of the game
10    wire [15:0][15:0] next_state;
11    // Load initial state into current_state on the next clock edge
12    always @(posedge clk) begin
13        if (load) begin
14            for (int i = 0; i < 16; i++) begin
15                current_state[i] = data[i*16 +: 16];
16            end
17        end
18    end
19
20    // Calculate the next state of the game every clock cycle
21    always @(posedge clk) begin
22        for (int i = 0; i < 16; i++) begin
23            for (int j = 0; j < 16; j++) begin
24                // Count the number of live neighbors
25                int live_neighbors = current_state[(i+15)%16][(j+15)%16] +
                    current_state[(i+15)%16][j] +
                    current_state[(i+15)%16][(j+1)%16] +
                    current_state[i][(j+15)%16] + current_state[i][(j+1)%16] +
                    current_state[(i+1)%16][(j+15)%16] +
                    current_state[(i+1)%16][j] +
                    current_state[(i+1)%16][(j+1)%16];
26                // Determine the next state of the cell based on the number
                of live neighbors
27                if (live_neighbors <= 1 || live_neighbors >= 4) begin
28                    next_state[i][j] = 0;
29                end else if (live_neighbors == 3) begin
30                    next_state[i][j] = 1;
31                end else begin
32                    next_state[i][j] = current_state[i][j];
33                end
34            end
35        end
36    end
37
38    // Update current_state with the next state
39    for (int i = 0; i < 16; i++) begin
40        for (int j = 0; j < 16; j++) begin
41            current_state[i][j] = next_state[i][j];
42        end
43    end
44
45    // Assign the current state to the output q
46    assign q = {current_state[0], current_state[1], current_state[2],
47        current_state[3], current_state[4], current_state[5], current_state[6],
48        current_state[7], current_state[8], current_state[9],
49        current_state[10], current_state[11], current_state[12],
50        current_state[13], current_state[14], current_state[15]};
51 endmodule

```

CodeGen-16B-FT:

```

1 module top_module(
2     input clk,
3     input load,
4     input [255:0] data,
5     output [255:0] q );
6     reg [255:0] q;
7     reg [255:0] next_q;
8     always @(posedge clk) begin
9         if (load) begin
10             q <= data;
11         end else begin
12             next_q <= q;
13             for (int i = 0; i < 16; i++) begin
14                 for (int j = 0; j < 16; j++) begin
15                     next_q[i*16+j] <= (q[i*16+j] == 1) ? (count_neighbors(i,
16                         j) == 2) : (q[i*16+j]);
17                 end
18             end
19             q <= next_q;
20         end
21     end
22     function int count_neighbors(int i, int j);
23     int count = 0;
24     for (int di = -1; di <= 1; di++) begin
25         for (int dj = -1; dj <= 1; dj++) begin
26             if (i+di >= 0 && i+di < 16 && j+dj >= 0 && j+dj < 16) begin
27                 if (q[i+di*16+j+dj] == 1) begin
28                     count++;
29                 end
30             end
31         end
32     end
33     return count;
34 end
endmodule

```

Fig. 20. GPT-3.5-turbo vs. CodeGen-16B-FT at implementing Conway game of life problem, category: Cellular Automata, Table 4. We highlight the mistakes .

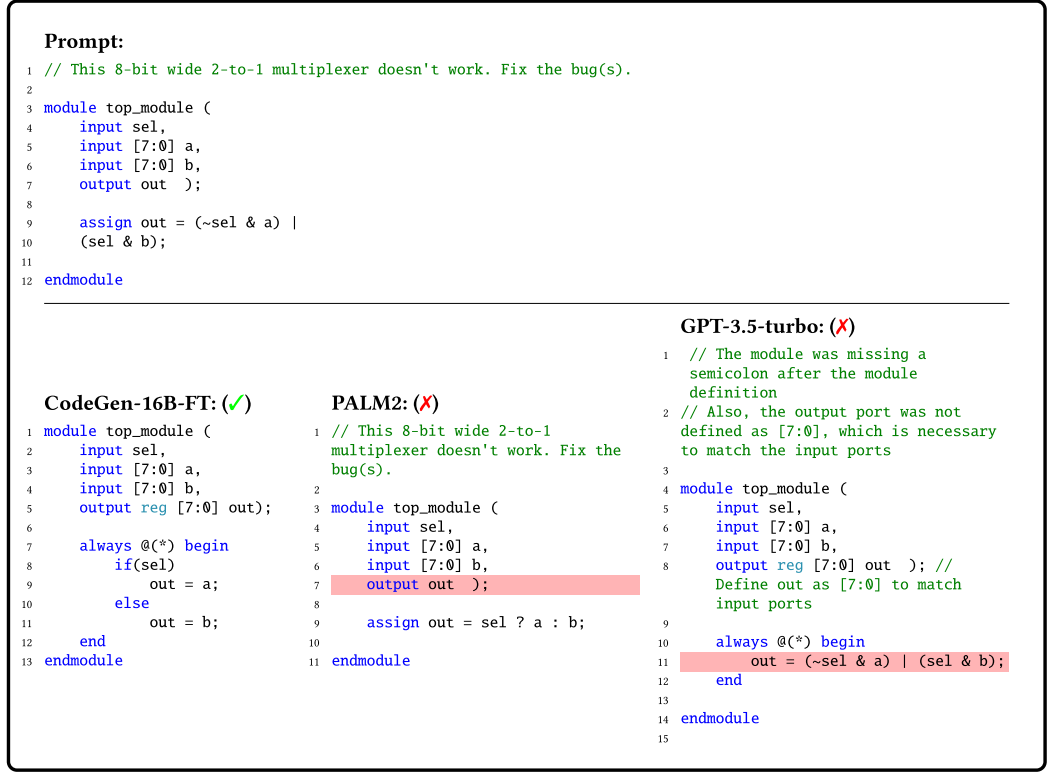


Fig. 21. CodeGen-16B-FT vs. GPT-3.5-turbo vs. PALM2 at fixing bug in 8-bit 2-to-1 MUX, category: Finding Bugs in Code, Table 4. We highlight the **mistake**.

to cater to a specific language. Refined CodeGen-16B LLM is the most proficient in generating functionally correct completions. On the whole, it delivered functionally accurate code 41.9% of the time, while the commercial top-tier (non-fine-tuned) code-davinci-002 LLM yielded functionally correct code 35.4% of time.

New entrant, GPT4 has shown outstanding performance across the problems, particularly it did well on advanced problem-solves. Smaller, fine-tuned LLMs like CodeGen-16B-FT are proficient and adaptable across a variety of problem complexities. Choosing an LLM is a balanced decision, considering use-case requirements, computational resources, problem complexity, cost, and reliability, particularly for commercial LLMs.

Despite its impressive performance, there is room for improvement for CodeGen-16B-FT. Future work could refine its fine-tuning process, perhaps through targeted training data or additional iterations. Exploring hybrid approaches that combine the strengths of different LLMs is another potential avenue. There is the prospect of integrating domain-specific knowledge into the model to enhance its understanding of Verilog code generation. With these enhancements, CodeGen-16B-FT could compete with larger LLMs like GPT4 in terms of efficiency and output quality.

REFERENCES

- [1] Aakash Ahmad, Muhammad Waseem, Peng Liang, Mahdi Fahmideh, Mst Shamima Aktar, and Tommi Mikkonen. 2023. Towards Human-Bot Collaborative Software Architecting with ChatGPT. In *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering (Oulu, Finland) (EASE'23)*. Association for Computing Machinery, New York, NY, USA, 279–285. <https://doi.org/10.1145/3593434.3593468>

- [2] Baleegh Ahmad, Shailja Thakur, Benjamin Tan, Ramesh Karri, and Hammond Pearce. 2024. On Hardware Security Bug Code Fixes By Prompting Large Language Models. *IEEE Transactions on Information Forensics and Security* (2024). 1–1. <https://doi.org/10.1109/TIFS.2024.3374558>
- [3] AI21. 2021. Jurassic-1 Language Models - AI21 Studio Docs. Retrieved November 2022 from <https://studio.ai21.com/docs/jurassic1-language-models/#general-purpose-models>
- [4] Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avizienis, John Wawrzynek, and Krste Asanović. 2012. Chisel: Constructing hardware in a scala embedded language. In *Proceedings of the 49th Annual Design Automation Conference (DAC '12)*. Association for Computing Machinery, New York, NY, USA, 1216–1225. DOI: <https://doi.org/10.1145/2228360.2228584>
- [5] Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. 2023. Chip-chat: Challenges and opportunities in conversational hardware design. In *Proceedings of the 2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*. IEEE. DOI: <https://doi.org/10.1109/mlcad58807.2023.10299874>
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*. H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin (Eds.), Vol. 33, Curran Associates, Inc., 1877–1901. Retrieved from <https://proceedings.neurips.cc/paper/2020/file/1457c0d6fbcb4967418bfb8ac142f64a-Paper.pdf>
- [7] Kaiyan Chang, Ying Wang, Haimeng Ren, Mengdi Wang, Shengwen Liang, Yinhe Han, Huawei Li, and Xiaowei Li. 2023. ChipGPT: How far are we from natural language hardware design. arXiv:2305.14019. Retrieved from <https://arxiv.org/abs/2305.14019>
- [8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374. Retrieved from <https://arxiv.org/abs/2107.03374>
- [9] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pilla, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. PaLM: Scaling Language Modeling with Pathways. arXiv:2204.02311. Retrieved from <https://arxiv.org/abs/2204.02311>
- [10] Ghada Dessouky, David Gens, Patrick Haney, Garrett Persyn, Arun Kanuparthi, Hareesh Khattri, Jason M. Fung, Ahmad-Reza Sadeghi, and Jeyavijayan Rajendran. 2019. HardFails: Insights into software-exploitable hardware bugs. 213–230. Retrieved from <https://www.usenix.org/conference/usenixsecurity19/presentation/dessouky>
- [11] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, Minneapolis, Minnesota, 4171–4186. DOI: <https://doi.org/10.18653/v1/N19-1423>
- [12] Rolf Drechsler, Ian G. Harris, and Robert Wille. 2012. Generating formal system models from natural language descriptions. In *Proceedings of the IEEE International High Level Design Validation and Test Workshop (HLDVT)*. 164–165.
- [13] Yonggan Fu, Yongan Zhang, Zhongzhi Yu, Sixu Li, Zhifan Ye, Chaojian Li, Cheng Wan, and Yingyan Celine Lin. 2023. GPT4AIGChip: Towards Next-Generation AI Accelerator Design Automation via Large Language Models. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 1–9. <https://doi.org/10.1109/ICCAD57390.2023.10323953>

- [14] Philip Gage. 1994. A new algorithm for data compression. *C Users Journal* 12, 2 (Feb. 1994), 23–38.
- [15] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. The Pile: An 800GB Dataset of Diverse Text for Language Modeling. arXiv:2101.00027. Retrieved from <https://arxiv.org/abs/2101.00027>
- [16] Christopher B. Harris and Ian G. Harris. 2016. GLaST: Learning formal grammars to translate natural language specifications into hardware assertions. In *Proceedings of the 2016 Design, Automation & Test in Europe Conference & Exhibition*. 966–971.
- [17] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2020. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. arXiv:1909.09436. Retrieved from <https://arxiv.org/abs/1909.09436>
- [18] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2023. StarCoder: May the source be with you! arXiv:2305.06161. Retrieved from <https://arxiv.org/abs/2305.06161>
- [19] Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. arXiv:2309.07544. Retrieved from <https://arxiv.org/abs/2309.07544>
- [20] Rada Mihalcea, Hugo Liu, and Henry Lieberman. 2006. NLP (natural language processing) for NLP (natural language programming). In *Computational Linguistics and Intelligent Text Processing*. Alexander Gelbukh (Ed.), Springer, Berlin, 319–330.
- [21] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. A Conversational Paradigm for Program Synthesis. arXiv:2203.13474. Retrieved from <https://arxiv.org/abs/2203.13474>
- [22] OpenAI. 2023. GPT-4. Retrieved 24 June 2023 from <https://openai.com/research/gpt-4>
- [23] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the keyboard? Assessing the security of Github Copilot’s code contributions. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP)*. 754–768. DOI: <https://doi.org/10.1109/SP46214.2022.9833571> ISSN: 2375-1207.
- [24] Hammond Pearce, Benjamin Tan, and Ramesh Karri. 2020. DAVE: Deriving automatically Verilog from English. In *Proceedings of the 2020 ACM/IEEE Workshop on Machine Learning for CAD*. ACM, 27–32. DOI: <https://doi.org/10.1145/3380446.3430634>
- [25] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. (2019), 24. Retrieved from https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
- [26] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-infinity: breaking the GPU memory wall for extreme scale deep learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC’21)*. Association for Computing Machinery, New York, NY, USA, Article 59, 14 pages. <https://doi.org/10.1145/3458817.3476205>
- [27] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD ’20)*. Association for Computing Machinery, New York, NY, USA, 3505–3506. DOI: <https://doi.org/10.1145/3394486.3406703>
- [28] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. ZeRO-Offload: Democratizing Billion-Scale Model Training. arXiv:2101.06840. Retrieved from <https://arxiv.org/abs/2101.06840>
- [29] Gustavo Sandoval, Hammond Pearce, Teo Nys, Ramesh Karri, Siddharth Garg, and Brendan Dolan-Gavitt. 2023. Lost at C: A user study on the security implications of large language model code assistants. In *Proceedings of the USENIX Security Symposium*.
- [30] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2020. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053. Retrieved from <https://arxiv.org/abs/1909.08053>
- [31] Shailja Thakur, Baleegh Ahmad, Zhenxing Fan, Hammond Pearce, Benjamin Tan, Ramesh Karri, Brendan Dolan-Gavitt, and Siddharth Garg. 2023. Benchmarking large language models for automated Verilog RTL code generation.

- In *Proceedings of the 2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1–6. DOI: <https://doi.org/10.23919/DATE56975.2023.10137086> ISSN: 1558-1101.
- [32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the International Conference on Neural Information Processing Systems*. Vol. 30, Curran Associates, Inc. Retrieved from <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [33] Stephen Williams. 2023. The ICARUS Verilog Compilation System. Retrieved 27 June 2023 from <https://github.com/steveicarus/iverilog> original-date: 2008-05-12T16:57:52Z.
- [34] Henry Wong. 2019. Project:About - HDLBits. Retrieved 25 June 2023 from <https://hdlbits.01xz.net/wiki/Project:About>
- [35] Ziqi Yan, Jiqiang Liu, Gang Li, Zhen Han, and Shuo Qiu. 2017. PrivMin: Differentially Private MinHash for Jaccard Similarity Computation. arXiv:1705.07258. Retrieved from <https://arxiv.org/abs/1705.07258>
- [36] Junjie Ye, Xuanning Chen, Nuo Xu, Can Zu, Zekai Shao, Shichun Liu, Yuhua Cui, Zeyang Zhou, Chao Gong, Yang Shen, Jie Zhou, Siming Chen, Tao Gui, Qi Zhang, and Xuanjing Huang. 2023. A Comprehensive Capability Analysis of GPT-3 and GPT-3.5 Series Models. arXiv:2303.10420. Retrieved from <https://arxiv.org/abs/2303.10420>

Received 27 June 2023; revised 24 December 2023; accepted 14 January 2024